AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 2: Building AI Agent Skills

Building AI Agent Skills: Best Practices and Common Limitations

A checklist for writing Skills that hold up, and the limits to plan around

In this chapter you turned the Orbit 2.0 launch review prompt into the `launch-risk-review` Skill, tested it for consistency, reused it across different tasks, and found where it stops. This article collects the lessons from those videos into practices you can apply to your own Skills.

What you built in this chapter

- **Video 1:** a 40-line prompt became a one-line request plus a Skill with three files: `SKILL.md`, `rubric.md`, and `report-template.md`.
- **Video 2:** the Skill produced the same structure and applied the same standard across different weeks and different wording.
- **Video 3:** the same Skill handled a go/no-go check, a vendor email, and a full multi-document review without changes.
- **Video 4:** the Skill couldn't answer questions about live tickets, recent changes, or the shared risk register, because it had no access to them.

Best practices

Separate know-how from context. Before writing a Skill, go through the prompt and ask of each line: will this change next time? What changes stays in the request. What doesn't is a candidate for the Skill.

Write the description for the agent. The agent uses the `description` to decide when the Skill applies. Describe the capability, then list the situations people will actually bring to it, in their own words. In the launch review Skill, "go/no-go recommendation" and "assess how an update affects the launch date" are why the vendor email request picked it up.

Keep SKILL.md short and point to detail. Put the process in `SKILL.md` and move detailed rules, templates, and reference material into supporting files. The agent reads them only when it needs them, which keeps its context small.

Make every rule testable. "More than two weeks of delay is High" gives the agent something to check. "Significant delay" doesn't. If two people could read a rule differently, the agent can too.

Write down the judgment, not just the steps. Rules like “prefer the most recent source,” “combine duplicates,” and “don’t invent facts, list what’s missing” are what an experienced teammate knows. They’re often missing from the original prompt.

Give the output a template. Exact headings, columns, and status rules are what make results comparable from run to run and person to person.

Keep one capability per Skill. A launch risk Skill and a launch announcement Skill, not a single Skill for everything launch-related.

Test like a team would. Run the Skill on more than one input, phrase the request in different ways, and clear the conversation between runs. Also try requests it should ignore, to check its description isn’t too broad.

Treat drift as a signal. When two runs disagree on something that matters, tighten the rule, the template, or the description, then test again.

Manage Skills like code. Keep them in the repository, review changes, and share them with the team from one place. That’s where the maintenance benefit comes from.

Common limitations

- **No access to external systems.** A Skill can’t see data that isn’t in the conversation or the workspace, such as tickets in a support desk.
- **Only as fresh as the last paste.** The agent’s knowledge stops at the newest document someone provided.
- **No actions elsewhere.** A Skill alone can’t create or update anything in another system, like the team’s risk register.
- **Consistent, not identical.** A Skill standardizes the structure, rules, and evidence. The exact wording still varies between runs.
- **Not for general preferences.** Broad, always-on behavior like “keep answers concise” belongs in the agent’s general instructions, not a Skill.

The first three limitations have the same root cause: the workflow needs access, not better instructions. That’s what the next chapter addresses with MCP.

Review your own Skill

Use these questions on a Skill you’re writing or one your team already uses:

1. Does the description say what the Skill does and list the situations it applies to?
2. Is `SKILL.md` short, with detailed rules and templates in separate files?
3. Could two people read any rule differently?
4. Is there an exact template for the output?
5. Does the Skill say what to do when information is missing or sources disagree?
6. Have you tested it on several inputs, several phrasings, and at least one request it should ignore?
7. Does any part of the workflow depend on information or actions outside the agent’s reach?

If the answer to the last question is yes, keep the Skill, and plan for access as well.